

Security in Software Development

IT-Security | 20.03.2026

Gerald Dißbauer | Lecturer

A-SIT Secure Information Technology Center Austria

www.dhbw-stuttgart.de

Agenda

Introduction

Fundamental Principles of Defensive Programming

Basic Security Patterns to Design Secure Architectures

Security Testing and Maintenance

Summary

Agenda

Introduction

Fundamental Principles of Defensive Programming

Basic Security Patterns to Design Secure Architectures

Security Testing and Maintenance

Summary

WHERE IN THE SECURE SOFTWARE LIFECYCLE ARE WE? IT DEPENDS ON THE APPLICABLE LIFECYCLE MODEL

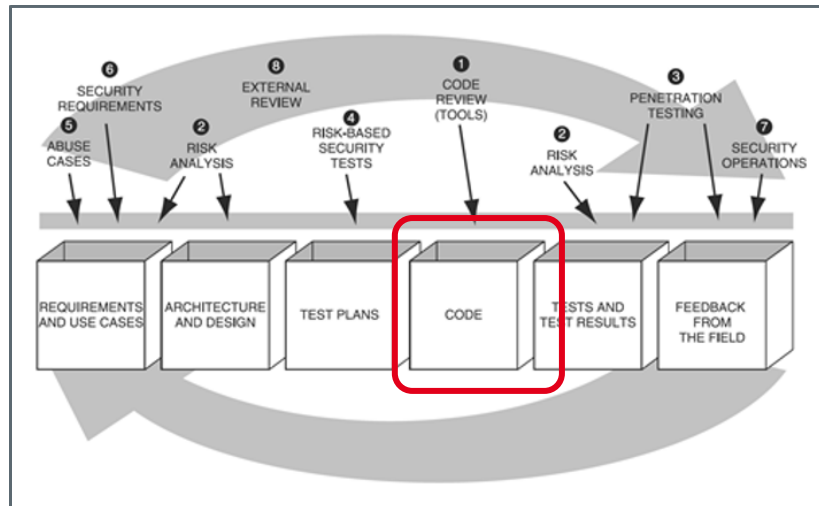
Recap from previous lectures (*3 selected Lifecycle models*)

- 7+1 Digital Touchpoints
- NIST SP 800-64
- SGP SDLC Model

SDL-models are applied both for secure and safe programming

CIGITAL SECURITY TOUCHPOINTS¹

7+1 TOUCHPOINTS



7 ORDERED SECURITY TOUCHPOINTS¹

- (1) Code Review
- (2) Architectural risk analysis
- (3) Penetration testing
- (4) Risk-based security tests
- (5) Abuse cases
- (6) Security requirements
- (7) Security operations
- (8) * External analysis^{2,3}

Effectiveness

- Apply security engineering best practices to artifacts and adopt the touchpoints
- Obtain priorities and decide the touchpoint ranking to remove bugs and flaws

¹ cf. (McGraw, 2006, p. 84)

² Annotation: External analysis is important. However, it is not a particular touchpoint, but is mentioned in the big-picture of the touchpoints

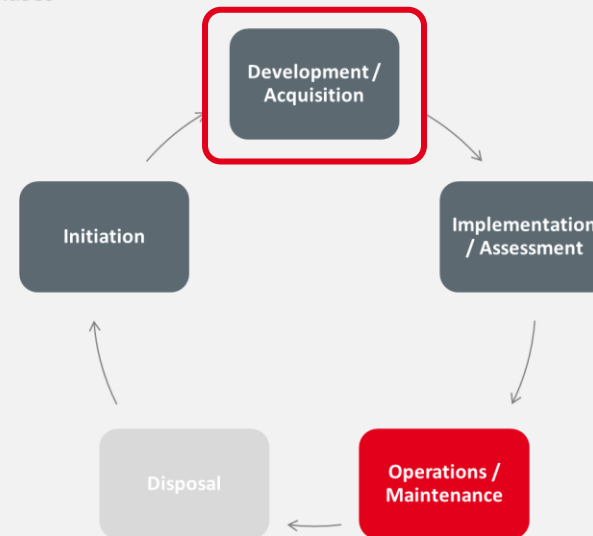
³ cf. (McGraw, 2006, pp. 88-89)

NIST¹ SP² 800-64

SECURITY IN THE SYSTEM DEVELOPMENT LIFECYCLE

- **Applicable independently**
- Custom built system
- Made from scratch
- Purchased and customized
- COTS³ are covered
- **SDLC**
- **Security Considerations**

Exhibit | 5 SDLC-Phases



Source: NIST SP 800-64 | Software-Development-Lifecycle – Own figure

¹ NIST – National Institute of Standards and Technology

² SP – Special Publication

³ COTS – Commercial off-the-shelf (software)

SGP¹ SDLC MODEL BY ISF²

2 SEPARATE PHASES AND 10 PROCESSES

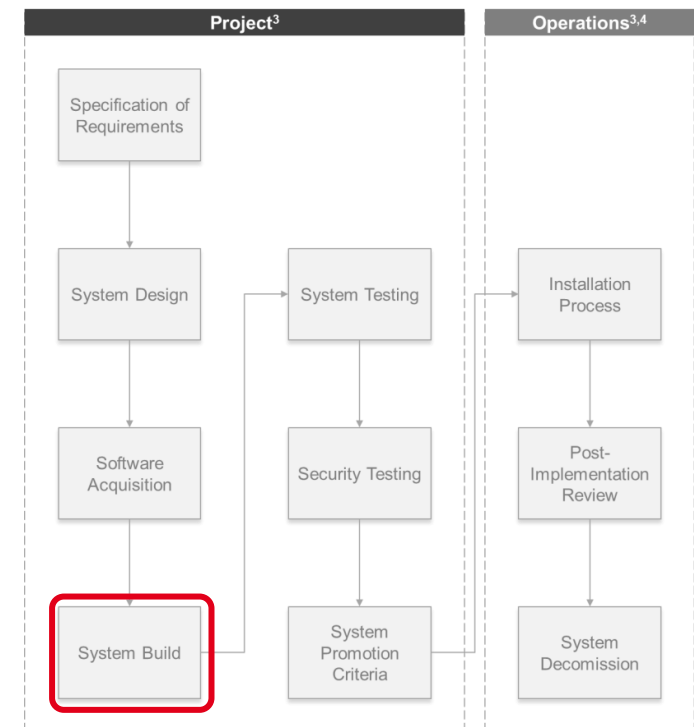
- Finer grained than NIST SP 800-64 model
- **Similar basic concepts**
- **Hardware** and/or **Software**
- **Supply chain considered**
- **Best practices** for development
- Consider information lifecycle

¹ SGP – Standard of Good Practice for Information Security

² ISF – Information Security Forum

³ Annotation: SGP model amended by G. Dißauer based on project experiences with two distinct phases (*i.e. Project, Operations*)

⁴ Annotation: Operations typically can include also e.g.: maintenance, (*emergency*) change management, system disposal



Own Figure based on (Stallings, 2019)

Agenda

Introduction

Fundamental Principles of Defensive Programming

Basic Security Patterns to Design Secure Architectures

Security Testing and Maintenance

Summary

SECURE PROGRAMMING AND PROGRAMMING VARIES THE DIFFERENCES AT A GLANCE

In general: The Developer/Company is responsible for providing secure and safe software with minimum or no known vulnerabilities

Safe Programming: Software should not cause **damage to the environment**

- Privilege escalation
- Attack something else
- Include Backdoors

Secure Programming: Software should be protected from all kinds of deficits¹, resistant against certain attacks or something that can cause harm to the software itself or to a system using it.

¹ Annotation: Deficits of software products can be segmented in: Failure, Fault, Error, Vulnerabilities

SOFTWARE PRODUCTS ARE NOT FREE FROM DEFICITS

FAILURE, FAULT, ERROR, VULNERABILITY

4 deficits of software products

Failure *(a deviation of a software product w.r.t. its required functions in required quality specified in functional and non-functional requirements – an externally observable reliability concept e.g. by end-user)*

Fault *(a failure has its origin in a fault of the software product e.g. an incorrect step, process, or definition in a software product – an internally observable reliability concept; alternatively “bug” or “defect”)*

Heads up: *Fault masking is the presence, when one or more defects hide other defect(s)*

Error *(reason for the presence of a fault that was caused by e.g. programmer, requirements engineer)*

Vulnerabilities *(i.e., particular errors that can be exploited by an adversary to gain)*

SECURE CODING PRINCIPLES

WHAT IS REQUIRED TO PROTECT DATA?

Security Control	The way to go by
Authentication	▪ Implement password security and account lockout for unsuccessful authentication attempts ▪ Consider unsecure „I forgot my password“ mechanisms ▪ Enforce POST methods for web applications
Authorization	▪ Verify authorization to gain access (for functions, pages etc.) ▪ Check the access context ▪ Validate security mechanisms on the server (client and server)
Cryptography	▪ Do not write your own crypto and use established libraries (and hardware as well) instead ▪ Encrypt both authentication data and payload data assets ▪ Hash data (e.g., passphrases) with salt and seed random number generators, appropriately

SECURE CODING PRINCIPLES

WHAT IS REQUIRED TO USE SERVICES AND DATA?

Security Control

The way to go by

Error Handling

- Detect and process deficits and implement exception handling
- Keep secret data private (also in logfiles) and do not disclose data, unnecessarily
- Cleanup error states and error conditions

Session Management

- Ensure protected communication channels and enforce a reasonable session lifespan
- Decide when session management is necessary and leverage existing solutions (e.g., TCP)
- Force a change of sessionID after a successful login (client and server)

Logging

- Avoid logging sensitive data (e.g., credentials, passphrases) and prepare for log spoofing
- Minimize the amount of data to be logged to the absolute necessity
- Increase the quality of logged data and reduce data quality issues

Data Validation

- Endorse the length of data before writing it to buffers
- Validate data before using (insert, output, take, accept)
- Verify data before sending/broadcasting to clients

GUIDING PRINCIPLES FOR SOFTWARE SECURITY

10 CONCRETE GUIDING PRINCIPLES^{1,2}

- (1) Secure the weakest link
- (2) Practice defense in depth
- (3) Fail securely
- (4) Follow the principle of least privilege
- (5) Compartmentalize
- (6) Keep it simple (also „economy of mechanism“)
- (7) Promote privacy
- (8) Remember that hiding secrets is hard
- (9) Be reluctant to trust
- (10) Use your community resources

¹ (Viega, McGraw, 2002, p. 92)

² Annotation: Can be compared with Saltzer and Schroeder principles

SECURE PROGRAMMING VS. SECURE ARCHITECTURES

THE MAIN IMPACTS

Programming errors and architecture security

- Programming errors (*e.g., deficits*) can imply that a secure architecture fails (*e.g. wrong evaluation of certificates*)
- A software without deficits based upon programming errors can have weaknesses in terms of software architecture

What are the most common programming errors?

SECURE PROGRAMMING

MOST COMMON ERRORS, AND GUIDELINES

Most common programming errors

- OWASP Top 10 vulnerabilities
- CWE/SANS „*Top 25 Most Dangerous Software Errors*“

Coding Guidelines

- CERT secure coding standards
- Secure coding guidelines for languages (*e.g., Java*)

THE COMMON WEAKNESS ENUMERATION (CWE)

CODING-LEVEL AND SECURITY-RELATED MISTAKES

Established project to list (to-date 891) weaknesses

- A MITRE-run initiative to **enumerate and provide standard identifiers for different coding-level security-related mistakes** that developers regularly make.
- A **CWE describes a weakness, but not vulnerabilities**¹. Exploitable weaknesses are vulnerabilities.
- Standard identifiers enable security guys to **share information about weaknesses** and for **tools to report findings** in a way that review teams can easily grasp.
- There is a lot of description information on weaknesses, which is beneficial to both the **reviewer and the developer**. The reviewer doesn't have to spend time duplicating a description that has been used many times over, and the developer doesn't have to rely on the communication skills of the developer.
- Static analysis tools use CWEs to report the weaknesses they detect.

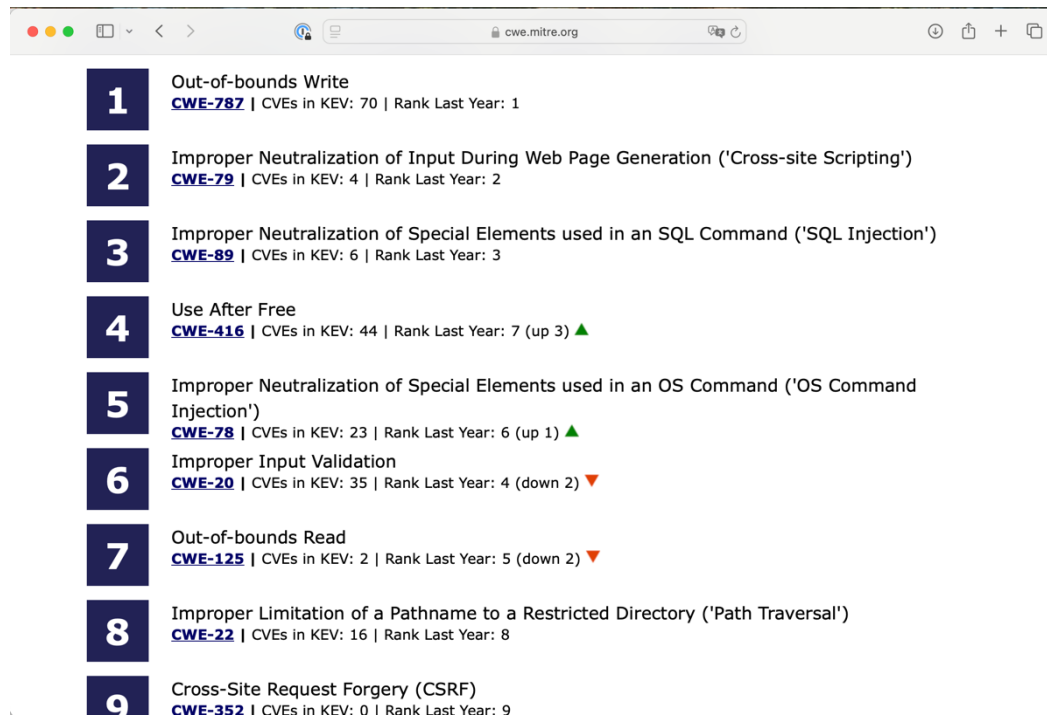
¹ Annotation: Known software vulnerabilities are described in so-called CVE's (Common Vulnerabilities and Exposures).

WHAT ARE THE CURRENT TOP 25 CWE?

LISTING FROM MITRE.ORG

Status: 24.10.2024

- CWE change over time
- Highest rank CWE-787¹
- Lowest rank CWE-79²



1	Out-of-bounds Write CWE-787 CVEs in KEV: 70 Rank Last Year: 1
2	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') CWE-79 CVEs in KEV: 4 Rank Last Year: 2
3	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') CWE-89 CVEs in KEV: 6 Rank Last Year: 3
4	Use After Free CWE-416 CVEs in KEV: 44 Rank Last Year: 7 (up 3) ▲
5	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection') CWE-78 CVEs in KEV: 23 Rank Last Year: 6 (up 1) ▲
6	Improper Input Validation CWE-20 CVEs in KEV: 35 Rank Last Year: 4 (down 2) ▼
7	Out-of-bounds Read CWE-125 CVEs in KEV: 2 Rank Last Year: 5 (down 2) ▼
8	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') CWE-22 CVEs in KEV: 16 Rank Last Year: 8
9	Cross-Site Request Forgery (CSRF) CWE-352 CVEs in KEV: 0 Rank Last Year: 9

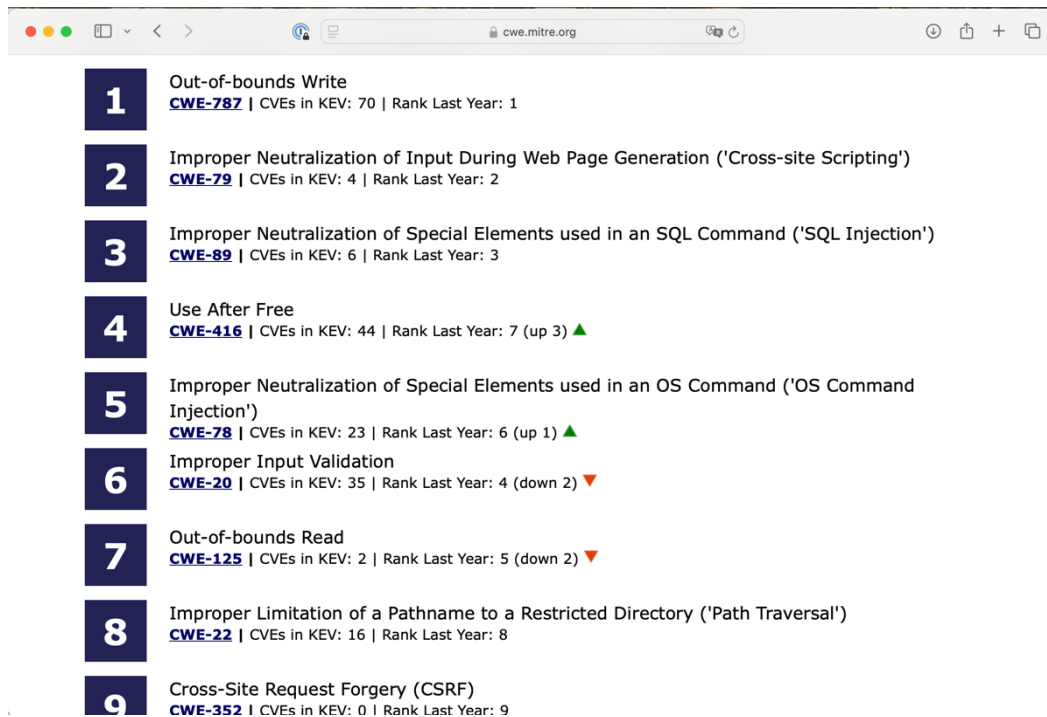
¹ Annotation: Out of bound writes.

² Annotation: Improper Neutralization of Input During Web Page Generation (XSS).

THE COMMON WEAKNESS ENUMERATION (CWE)

CODING-LEVEL AND SECURITY-RELATED MISTAKES

- Unique identifier
- Database
- Top 25 CWEs retrievable

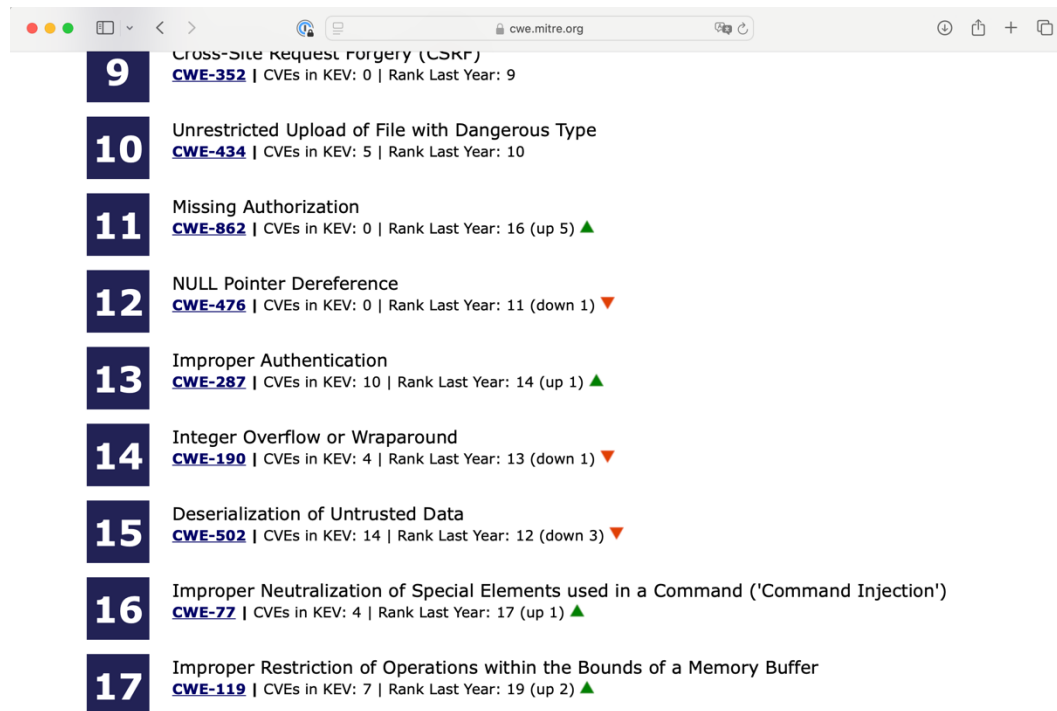


1	Out-of-bounds Write CWE-787 CVEs in KEV: 70 Rank Last Year: 1
2	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') CWE-79 CVEs in KEV: 4 Rank Last Year: 2
3	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') CWE-89 CVEs in KEV: 6 Rank Last Year: 3
4	Use After Free CWE-416 CVEs in KEV: 44 Rank Last Year: 7 (up 3) ▲
5	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection') CWE-78 CVEs in KEV: 23 Rank Last Year: 6 (up 1) ▲
6	Improper Input Validation CWE-20 CVEs in KEV: 35 Rank Last Year: 4 (down 2) ▼
7	Out-of-bounds Read CWE-125 CVEs in KEV: 2 Rank Last Year: 5 (down 2) ▼
8	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') CWE-22 CVEs in KEV: 16 Rank Last Year: 8
9	Cross-Site Request Forgery (CSRF) CWE-352 CVEs in KEV: 0 Rank Last Year: 9

THE COMMON WEAKNESS ENUMERATION (CWE) | cont'd

CODING-LEVEL AND SECURITY-RELATED MISTAKES

- Unique identifier
- Database
- Top 25 CWEs retrievable

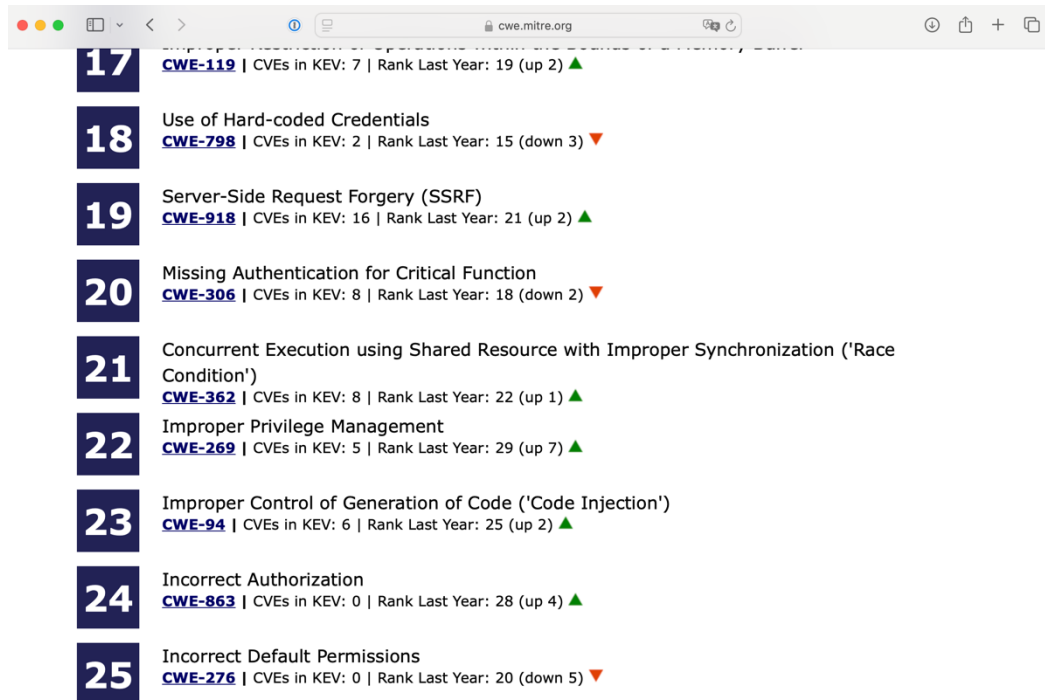


9	Cross-Site Request Forgery (CSRF) CWE-352 CVEs in KEV: 0 Rank Last Year: 9
10	Unrestricted Upload of File with Dangerous Type CWE-434 CVEs in KEV: 5 Rank Last Year: 10
11	Missing Authorization CWE-862 CVEs in KEV: 0 Rank Last Year: 16 (up 5) ▲
12	NULL Pointer Dereference CWE-476 CVEs in KEV: 0 Rank Last Year: 11 (down 1) ▼
13	Improper Authentication CWE-287 CVEs in KEV: 10 Rank Last Year: 14 (up 1) ▲
14	Integer Overflow or Wraparound CWE-190 CVEs in KEV: 4 Rank Last Year: 13 (down 1) ▼
15	Deserialization of Untrusted Data CWE-502 CVEs in KEV: 14 Rank Last Year: 12 (down 3) ▼
16	Improper Neutralization of Special Elements used in a Command ('Command Injection') CWE-77 CVEs in KEV: 4 Rank Last Year: 17 (up 1) ▲
17	Improper Restriction of Operations within the Bounds of a Memory Buffer CWE-119 CVEs in KEV: 7 Rank Last Year: 19 (up 2) ▲

THE COMMON WEAKNESS ENUMERATION (CWE) | cont'd

CODING-LEVEL AND SECURITY-RELATED MISTAKES

- Unique identifier
- Database
- Top 25 CWEs retrievable



17	CWE-119 CVEs in KEV: 7 Rank Last Year: 19 (up 2) ▲
18	Use of Hard-coded Credentials CWE-798 CVEs in KEV: 2 Rank Last Year: 15 (down 3) ▼
19	Server-Side Request Forgery (SSRF) CWE-918 CVEs in KEV: 16 Rank Last Year: 21 (up 2) ▲
20	Missing Authentication for Critical Function CWE-306 CVEs in KEV: 8 Rank Last Year: 18 (down 2) ▼
21	Concurrent Execution using Shared Resource with Improper Synchronization ('Race Condition') CWE-362 CVEs in KEV: 8 Rank Last Year: 22 (up 1) ▲
22	Improper Privilege Management CWE-269 CVEs in KEV: 5 Rank Last Year: 29 (up 7) ▲
23	Improper Control of Generation of Code ('Code Injection') CWE-94 CVEs in KEV: 6 Rank Last Year: 25 (up 2) ▲
24	Incorrect Authorization CWE-863 CVEs in KEV: 0 Rank Last Year: 28 (up 4) ▲
25	Incorrect Default Permissions CWE-276 CVEs in KEV: 0 Rank Last Year: 20 (down 5) ▼

PENETRATE AND PATCH NOT SUFFICIENT IN SECURE CODING

- Developers can only patch known problems
- Attackers identify problems without reporting them to developers
- Patches are rushed out under pressure and introduce new problems
- Patches most likely fix the symptom and not the underlying problem
- Unapplied security patches are dangerous because the reality is often „never change a running system“
- Attackers derive details about vulnerabilities from patches
- Attacks are possible until a system is patched

!!! Security is not an add-on feature !!!

PASSWORD SECURITY REQUIREMENTS | CWE-521

CWE-521: Weak Password Requirements

Recommendations

- Minimum length enforcement
- Complex composition
- Must not contain the user name as a substring
- Users should be able to change the password
- Password expiration to be considered
- Prevent reuse of previous passwords when changed
- Avoid time-driven-only expiration of passwords

ATTACKS AGAINST PASSWORDS

RESISTANCE OF STRONG PASSWORDS

Relevant attacks against passwords

- Password guessing
- Brute force attacks
- Dictionary attacks
- **Replay attacks**

Password strength is the resistance against attacks

- Length
- Complexity
- Unpredictability

HOW IS REPLAY ATTACK PREVENTION POSSIBLE? PROTECTION REQUIRED

Encrypting and hashing of passwords does not sufficiently prevent from replay attacks against passwords

Sniffing valuable data (e.g., hashed passwords) and (re-)sends it to the server
Knowing the hash is sufficient

Proposed solution to prevent replay attacks against passwords

- Add context information to the message (*e.g., sessionID, time stamp*)
- Generate and transmit **One-time-Passwords** (*i.e., Tokens*)

FUNDAMENTALS ABOUT ONE-TIME-PASSWORDS

RFC 2289

- Passwords that are only used once (*OTPs*)
- Very often used in multi-factor environments (*e.g., digital signature systems*)
- Not vulnerable against replay attacks
- Solves the SSO-problem and the problem of using the same password for services

OTP generation

- Time synchronization between a client and a server (*i.e., short validity*)
- Previous passwords (*OTPs are predefined chains*)
- Challenge (*random number derived by a server or an arbitrary counter*)

INPUT AND OUTPUT VALIDATION

DO NOT TRUST UNTIL PROVEN OTHERWISE

Validate all input (*and output*) (e.g., CWE-79)

- Every input is bad until proven otherwise
- Lookup for data that matches expected use
- Reject all data that does not match expected use

Restrict, reject and sanitize

- Type checks (e.g., *numerics*)
- Length checks (e.g., *buffer < n byte*)
- Range checks (e.g., *A-Z, a-z, and/or forbidden symbols such as: i, l, 1 etc.*)
- Format checks

Agenda

Introduction

Fundamental Principles of Defensive Programming

Basic Security Patterns to Design Secure Architectures

Security Testing and Maintenance

Summary

SECURITY OBJECTIVES AND TRADEOFFS

IMPERFECT ARCHITECTURES

Security objectives: confidentiality, integrity, availability, authenticity, authorization
Security objectives can conflict with each other, when designing a secure software architecture, e.g.,

- Authenticity vs. integrity
(e.g., an attacker impersonates a user and modifies data, unauthorized)
- Availability vs. authenticity
(e.g., hash primitives with bad performance)
- Confidentiality vs. Availability
(e.g., encrypted data are recoverable, if decryption key lost)
- Integrity vs. confidentiality
(e.g., an attacker modifies encrypted data, unauthorized)

DESIGNING A SYSTEM OBTAIN AN ARCHITECTURE

Map security requirements by the design

A robust architecture is the basis for a secure system

Consider common security best practices

- Security patterns provide solutions for recurring problems in the security domain
- Attack patterns provide descriptions of attacks preconditions and enable to derive security aspects for the system to be developed
- Create UMLSec diagrams to model security requirements in UML¹

¹ UML – Unified Modeling Language

WHAT ARE SECURITY PATTERNS?

SOLUTIONS TO PROBLEMS

Security pattern. A solution to a problem that addresses a set of threats by implementing security controls.

Security patterns mitigate (or eliminate) a threat but explicitly do not fix a vulnerability.

Security patterns are directly related to threats (i.e., not to vulnerabilities)

EXAMPLES OF PATTERNS

SELECTED REFERENCES

Security patterns provide paradigms and/or samples for designing secure software components for both organizational and technical purposes

- Based on international best practices
- Reusable prototype
- Technical patterns (*e.g., reference monitor*)
- Organizational patterns (*e.g., documentation, installation, configuration*)

Retrievable patterns

- <https://www.owasp.org>
- <https://www.opensecurityarchitecture.org/>

SECURITY PATTERNS

4 CATEGORIES

- Architectural patterns

Describe software architecture concepts

- Design patterns

Code-oriented because security is considered as an aspect of a subsystem

- Analysis patterns

Implementation-free security mechanisms that to define system-independent requirements.

- Special type of patterns

Adding or removing new sections to standard templates by using special notations.

WHICH PARTICULAR SECURITY PATTERNS EXIST?

SELECTED EXAMPLES

- Identity Management
- Authentication
- Access Control
- Secure Process Management
- Secure Execution and File Management
- Secure OS Architecture and Administration
- Network
- Web Services Security
- Web Services Cryptography
- Secure Middleware
- Misuse Patterns
- Cloud Computing Architecture

IDENTITY MANAGEMENT SECURITY PATTERNS

SELECTED EXAMPLES

Security patterns for identity management

- Circle of Trust
- Identity Provider
- Identity Federation
- Liberty Alliance Identity Federation

IDENTITY MANAGEMENT SECURITY PATTERNS

SELECTED EXAMPLES

Security patterns for identity management

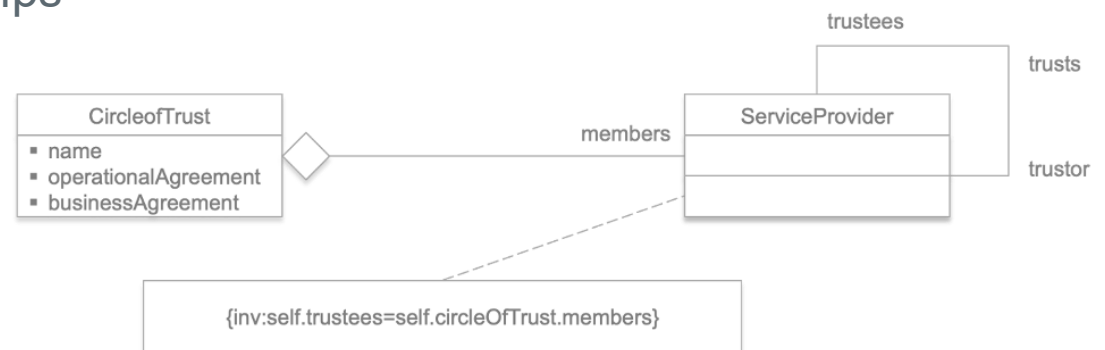
- **Circle of Trust**
- Identity Provider
- Identity Federation
- Liberty Alliance Identity Federation

THE CIRCLE OF TRUST PATTERN

TRUST RELATIONSHIP AMONG SERVICE PROVIDERS

The Circle of Trust pattern

- Formation of trust relationships
- Subjects obtain access
- Secure environment
- More secure interaction
- Set of service providers
- Federate identities of users



Own figure based on (Fernandez, 2013, p. 35)

One circle of trust includes several service providers

IDENTITY MANAGEMENT SECURITY PATTERNS

SELECTED EXAMPLES

Security patterns for identity management

- Circle of Trust
- **Identity Provider**
- Identity Federation
- Liberty Alliance Identity Federation

THE IDENTITY PROVIDER PATTERN

SUBJECTS IDENTITIES

The Identity provider pattern

- Centralization of identity information
- Security domain
- Propagate information to services and applications
- Solves the problem of managing identities
- Subjects and resources typically from same organization

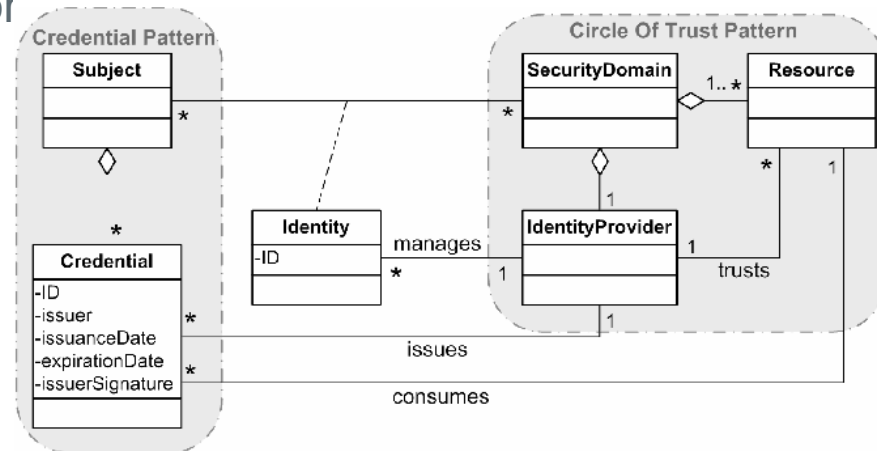


Figure 3. UML class diagram for the Identity Provider pattern

Figure taken from (Delessy, Fernandez, Larrondo-Petrie, 2007)

¹ Delessy, N., Fernandez, E.B., and Larrondo-Petrie, M. (2007). A Pattern Language for Identity Management. 2007 International Multi-Conference on Computing in the Global Information Technology (ICCGI'07), 31-31.

AUTHENTICATION SECURITY PATTERNS

SELECTED EXAMPLES

Security patterns for authentication

- Authenticator
- Remote Authenticator/Authorizer
- Credential

AUTHENTICATION SECURITY PATTERNS

SELECTED EXAMPLES

Security patterns for authentication

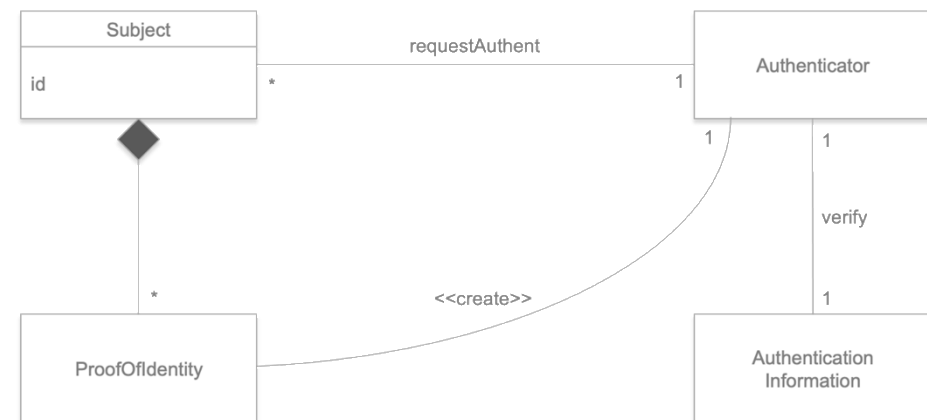
- **Authenticator**
- Remote Authenticator/Authorizer
- Credential

THE AUTHENTICATOR PATTERN

SUBJECT IDENTIFICATION AGAINST A SYSTEM

The authenticator pattern

- Preventing imposters accessing our system
- Legitimate users access a system
- Attackers can impersonate users
- Single point of access
- Apply a protocol to verify the identity of the legitimate users



Own figure based on (Fernandez, 2013, p. 54)

ACCESS CONTROL PATTERNS

SELECTED EXAMPLES

Access Control Patterns

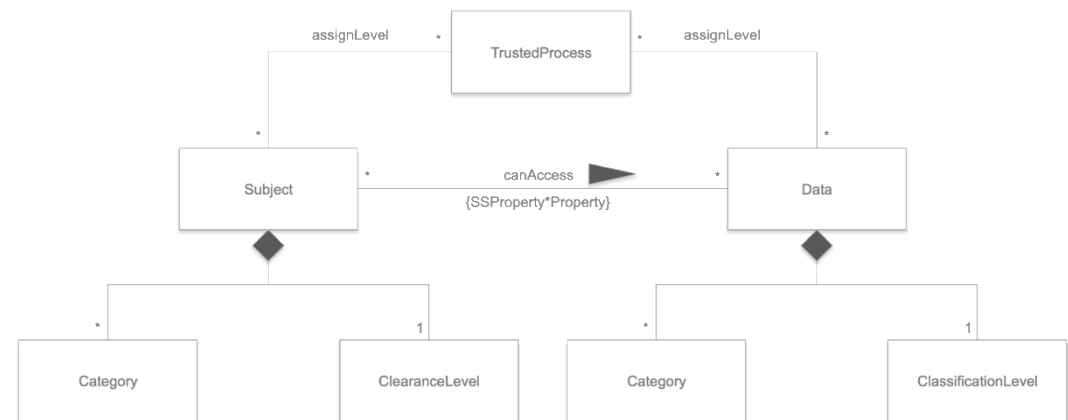
- Authorization
- Role-based Access Control
- **Multilevel Security**
- Policy-based Access Control
- Access Control List
- Capability
- Refied Reference Monitor
- Controlled Access Session
- Session-based Role-based Access Control
- Security Logger and Auditor

THE MULTILEVEL SECURITY PATTERN

CATEGORIZATION OF SENSITIVE DATA

The multilevel security pattern

- Assign classifications (*clearances*) to users
- Assign classifications (*sensitivity levels*) to data
- Separate units into categories
- Policies enable access to assets for users



Own figure based on (Fernandez, 2013, p. 83)

BRIEF OVERVIEW ABOUT OTHER SECURITY PATTERNS

SELECTED SECURITY PATTERNS

Categories

- (1) Secure Process Management
- (2) Secure Execution and File Management Patterns
- (3) Secure OS Architecture and Administration Patterns
- (4) Network Security Patterns
- (5) Web Services Security Patterns
- (6) Web Services Cryptography Patterns
- (7) Secure Middleware Patterns
- (8) Misuse Patterns
- (9) Cloud Computing Architecture Patterns

SECURE PROCESS MANAGEMENT PATTERNS

SELECTED EXAMPLES

Secure Process Management Patterns

- Secure Process/Thread
- Controlled-Process Creator
- Controlled Object Factory
- Controlled Object Monitor
- Protected Entry Points
- Protection Rings

SECURE EXECUTION AND FILE MANAGEMENT PATTERN

SELECTED EXAMPLES

Secure Execution and File Management Patterns

- Virtual Address Space Access Control
- Execution Domain
- Controlled Execution Domain
- Virtual Address Space Structure Selection

SECURE OS ARCHITECTURE AND ADMINISTRATION

SELECTED EXAMPLES

Secure OS Architecture and Administration Patterns

- Modular Operating System Architecture
- Layered Operating System Architecture
- Microkernel Operating System Architecture
- Virtual Machine Operating System Architecture
- Administrator Hierarchy
- File Access Control

NETWORK SECURITY PATTERNS

SELECTED EXAMPLES

Network Security Patterns

- Abstract Virtual Private Network
- IPSec VPN
- TLS Virtual Private Network
- Transport Layer Security
- Abstract IDS
- Signature-Based IDS
- Behavior-Based IDS

WEB SERVICES SECURITY PATTERNS

SELECTED EXAMPLES

Web Services Security Patterns

- Application Firewall
- XML Firewall
- XACML¹ Firewall
- XACML Authorization
- XACML Access Control Evaluation
- Web Services Policy Language
- WS-Policy
- WS-Trust
- SAML Assertion

¹ XACML – eXtensible Access Control Markup Language

² SAML – Security Assertion Markup Language

WEB SERVICES CRYPTOGRAPHY PATTERNS

SELECTED EXAMPLES

Web Services Cryptography Patterns

- Symmetric Encryption
- Asymmetric Encryption
- Digital Signature with Hashing
- XML¹ Encryption
- XML Signature
- WS-Security

¹ XML – Extended Markup Language

SECURE MIDDLEWARE PATTERNS

SELECTED EXAMPLES

Secure Middleware Patterns

- Secure Broker
- Secure Pipes and Filters
- Secure Blackboard
- Secure Adapter
- Secure Three-Tier Architecture
- Secure Enterprise Service Bus
- Secure Distributed Publish/Subscribe
- Secure Model-View-Controller

MISUSE PATTERNS

SELECTED EXAMPLES

Misuse Patterns

- Worm
- Denial-of-Service in VoIP¹
- Spoofing Web Services

¹ VoIP – Voice over IP (Internet Protocol)

CLOUD COMPUTING ARCHITECTURE PATTERNS

SELECTED EXAMPLES

Cloud Computing Architecture Patterns

- Infrastructure-as-a-Service
- Platform-as-a-Service
- Software-as-a-Service

Agenda

Introduction

Fundamental Principles of Defensive Programming

Basic Security Patterns to Design Secure Architectures

Security Testing and Maintenance

Summary

TAKE MALICIOUS INPUTS INTO ACCOUNT

SELECTED EXAMPLES OF POSSIBLE INPUTS

Examples of possible inputs

- Register settings
- Environment variables (*e.g., in python 'HOME', 'VERY_LONG_KEY_THAT_EXIST'*)
- Data of individuals to derive electronic certificates (*e.g. date of birth, place of birth, address*)
- Parameters of cryptographic algorithms (*e.g., key size*)
- Firmware and/or hardware versions (*e.g., HSM*)
- Cryptographic one-time token (*e.g., bound to one particular session/transaction*)
- DSA¹ Domain parameters ($p, q, g \{, domain_parameter_seed, counter\}$)
- PBKDF2² of PKCS³ #5 ($P, S, c, dkLen$)⁴
- EMSA⁵-PKCS1-v1_5-ENCODE ($M, emLen$)

¹ DSA – Digital Signature Standard

² PBKDF2 – Password-Based Key Derivation Function 2

³ PKCS – Public-Key Cryptography Standards

⁴ cf. (NWG PKCS #5, 2000)

⁵ EMSA – Encoding Method for Signature with Appendix

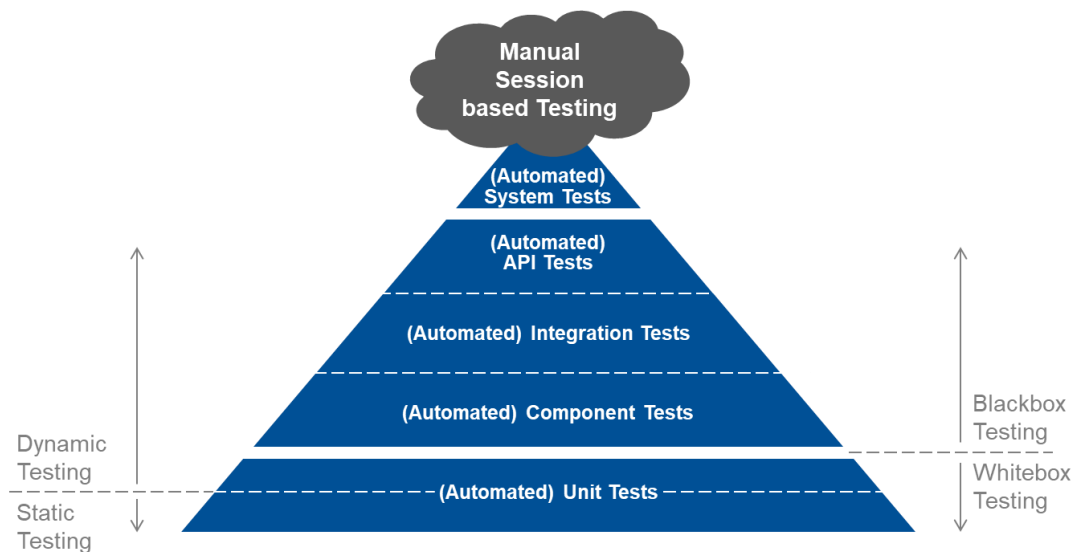
Can we show the absence of deficits in software products?

THE TEST PYRAMID^{1,2}

Detect errors early in the SDL and conduct testing quickly (e.g., parallel)

Basic test types in terms of risk based security testing

- (1) Functional testing
- (2) Non-functional testing
- (3) Structural testing
- (4) Change-related testing
- (5) Security testing
- (6) Environment testing



¹ Data Source: (Cohn, 2009)

² Data Source: (Witte, 2016)

Own figure based on (Cohn, 2009) and (Witte, 2016)

SECURITY TESTING VS. CONVENTIONAL TESTING

THE DIFFERENCES

Functional tests

- Specification as a basis
- Observe the expected output
- Tests are self-contained

Security tests

- Specification cannot be used as basis
- Exploring side-effects and unwanted behavior
- Tests affect each other

Tests are snapshots | Attackers have unlimited time

CLASSIFICATION OF TESTING

7 APPROACHES

White-box Testing

- Testing all implementations of the system under test (*SUT*)
- Path coverage considerations
- Faults of commission
- Find implementation flaws
- No guarantee that a specification is fulfilled

Black-box Testing

- Specifications are tested
- Input and output are relevant – no access to application internals
- Flaws in the specification are detected
- No guarantee that the implementation is correct

CLASSIFICATION OF TESTING | cont'd

7 APPROACHES

Gray-Box Testing

- A combination of both black-box testing and white-box testing
- Preferred testing technique
- Derive variants based on internal data (*White-box testing*)
- Conduct tests with data variants (*Black-box testing*)

CLASSIFICATION OF TESTING | cont'd

7 APPROACHES

Static testing

- Validate requirements and architecture
- Secure coding audit
- Reasoning about properties of the code
- No code execution

Dynamic testing

- Input and observe output during runtime
- Validate certain inputs and obtain outputs
- Code execution

CLASSIFICATION OF TESTING | cont'd

7 APPROACHES

Automatic testing

- Conduct continuous testing
- Compare input/output configurations

Regression testing

- Repeat testing
- Introducing new functionality does not compromise existing ones

WHEN CAN TESTING BE PERFORMED?

DIFFERENT LIFECYCLE PHASES

Testing should be performed in different lifecycle phases

- Requirements analysis phase
- Design phase
- Implementation phase
- Entry-into-operations phase

The applicable test method depends on the development phase

GENERAL OVERVIEW ABOUT TESTING IN SECURITY

WHY DO WE NEED TESTING?

When system is designed and implemented
correctness has to be *tested*

Different types of tests are necessary

Validation

- is the system designed correctly?
- does the design meet the problem requirements?

Verification

- is the system implemented correctly?
- does the implementation meet the design requirements?

Different features can be tested – functionality, performance, *security*

TEST-DRIVEN DEVELOPMENT

THE GENERAL APPROACH

Conventional software testing, is a quality assurance method. It aims to detect errors in already existing solutions of a software and system under test (*SuT*) for production systems in short cycles.

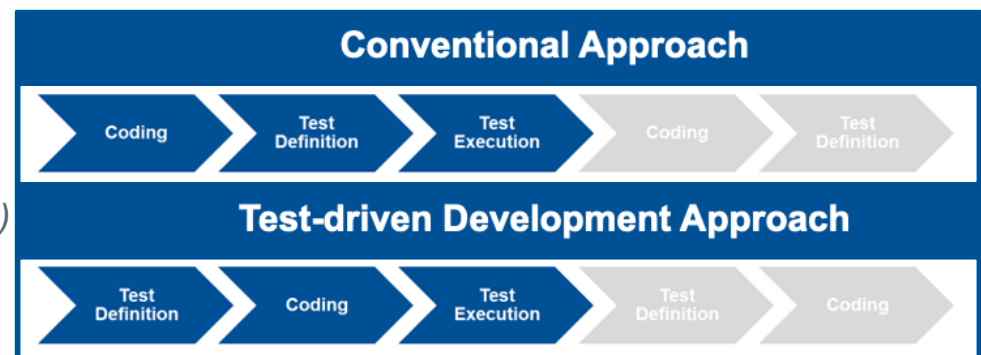
Test-driven development (*TDD*) is also known as „*test-first-development*“ or „*Test-first Programming*“

It is a software development practice where small sections of test code are used to direct the implementation of program units and it requires the writing of test code BEFORE the production code

- **Rule I**
Write new code if an automated test fails
- **Rule II**
Eliminate duplication

The order of coding tasks i.e. Lifecycle

- Think (*plan and select requirements*)
- **Red** (*implementation starts with a failing test*)
- **Green** (*make the code pass your test*)
- Refactor (*clean up your code*)



TEST-DRIVEN DEVELOPMENT | cont'd

SHOWING THE PRESENCE OF DEFICITS

The result of that iterative approach is to test the code until it passes the test cases without failure and integrate automatically tested code continuously, to obtain an increased confidence in the software product

Implication: Think and develop/write a test case, watch it fail, write simple code to make it pass

Each test (*as an executable specification*) should verify only one condition / thing

Avoid duplications (*TDD consumes a little bit more time than other approaches*)

Mirror your test cases in a directory to your production code

The asymmetry of testing in the creation of software products

“Program testing can be used to show the presence of deficits, but never to show their absence!”¹

¹ cf. (Dijkstra, 1970, p. EWD-249 7)

TDD-PERSPECTIVES FROM THE V-MODEL SOFTWARE-DEVELOPMENT MODEL

3 Views

- User view
(fielded production system, requirements, specifications)
- Architecture view
(architecture, specification, interfaces, interdependencies)
- Implementation view
(components, TDD-focus)



¹ Annotation: Low and/or high level means in terms of a level of abstraction where high has another quality in terms of precision

A SMALL TDD-EXAMPLE

10 SELECTED REQUIREMENTS

Objective: We want to write a function that obtains the current market capitalization of a publicly traded corporation. For simplicity, we elicit the following 10 requirements:

R-1: The current price of a single share outstanding cannot be negative
(if the price of the share is zero, the corporation files for bankruptcy)

R-2: The currency of the price of the single shares outstanding is USD

R-3: The number of shares outstanding cannot be negative
(if the number of shares is zero e.g. because it is the result of a massive stock-purchase program, the company is no longer publicly traded on the markets)

R-4: The current price of a single share outstanding can be either an integer or floating

R-5: The number of shares outstanding is a positive integer or it has no upper bound

R-6: *market capitalization* = *current share price* * *number* of shares outstanding

R-7: Detect input parameter errors in terms of type and value

R-8: Compute market capitalization only if input is valid both in terms of type and value

R-9: Construct output to visualize the results and print it to the screen

R-10: Raise an error message if either the type or the value of the input is invalid

A SMALL TDD-EXAMPLE

10 SELECTED REQUIREMENTS

```
# compute_market_cap.py
# source: GD

class ComputeMarketCapitalization(object):

    def multiply(self, currentStockPrice, NumberOfStocks):

        if (type(currentStockPrice) not in [int, float] or type(NumberOfStocks) not in [int]):
            raise TypeError("The number types are invalid.")
        else:
            if ((currentStockPrice < 0) or (NumberOfStocks < 0)):
                raise ValueError("The number values cannot be a negative number.")
            else:
                print('The current Stock Price is: {}'.format(currentStockPrice))
                print('The current number of outstanding shares is: {}'.format(NumberOfStocks))
                result = currentStockPrice * NumberOfStocks
                print('The current market capitalization is: {}\n'.format(result))
                return result
```

Agenda

Introduction

Fundamental Principles of Defensive Programming

Basic Security Patterns to Design Secure Architectures

Security Testing and Maintenance

Summary

SUMMARY

COVERED TOPICS

- Complexity is an enemy of security
- Profound architecture is the basis of secure software
- IT security objectives may conflict with another
- Select design principles and patterns w.r.t. business needs
- Balancing of different design principles is recommended
- Security modeling and security patterns offer concepts to design secure software components

